DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RV COLLEGE OF ENGINEERING®

(Autonomous Institution affiliated to VTU, Belagavi)

PROJECT REPORT

On

Master Algorithm: An Interactive Platform for Algorithm Execution Visualization

1RV24CY007	**Avishkar More**
1RV24CY017	**Giri Vardhan Santhosh Kumar**

Submitted byUnder the guidance of

Prof. Ganashree K C

Assistant Professor

2025 – 2026

RV COLLEGE OF ENGINEERING®, BENGALURU–560059

(*Autonomous Institution Affiliated to VTU, Belagavi*)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work titled **"Master Algorithm: An Interactive Platform for Algorithm Execution Visualization"** carried out by [Avishkar More (1RV24CY007)], [Giri Vardhan Santhosh Kumar (1RV24CY017)] in partial fulfilment of the requirement for the degree of **Bachelor of Engineering in Computer Science and Engineering (Cyber Security)** of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report.

Prof. Ganashree K C	Dr. Mrinal Moharir
Assistant Professor	Program Coordinator of CSE (Cyber Security)
Department of CSE	

External Examiners

Name of Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

Master Algorithm: An Interactive Platform for Algorithm Execution Visualization

ABSTRACT

Understanding algorithms is a fundamental requirement in computer science education, yet many learners find it difficult to visualize how algorithms execute internally due to the abstract nature of control flow, data movement, and state changes. Traditional teaching methods often rely on static diagrams and pseudocode, which fail to effectively demonstrate intermediate execution steps.

Master Algorithm addresses this challenge by providing an interactive platform that visualizes the step-by-step execution of algorithms using real implementations written in the C programming language. The system executes native algorithm code, captures runtime execution traces, and presents them through a modern web interface. The platform is built using a layered architecture comprising a React (Vite) frontend, a Node.js and Express backend, and independently compiled C programs.

The enhanced platform supports 100 algorithms across multiple categories, including sorting, searching, graph algorithms, data structures, dynamic programming, string matching, and computational complexity concepts. It introduces a redesigned categorized dashboard, interactive guided tutorials with MCQ-based checkpoints, integrated DAA theory, Interview Mode for technical interview preparation, customizable themes, and improved accessibility and execution reliability.

By combining real algorithm execution, interactive visualization, theoretical learning resources, and interview-oriented practice within a single platform, Master Algorithm bridges the gap between theoretical concepts and practical implementation, making it an effective learning tool for students, educators, self-learning, and technical interview preparation.

1. INTRODUCTION

1.1 Background

Algorithms form the foundation of computer science and play a vital role in solving computational problems efficiently. They define the logical sequence of operations through which data is processed, transformed, and analyzed in software systems. As students progress through computer science education, they encounter a wide range of algorithms involving sorting, searching, data structures, graph traversal, recursion, dynamic programming, and optimization techniques. However, understanding how these algorithms behave internally during execution remains a significant challenge for many learners.

Traditional methods of teaching algorithms primarily rely on theoretical explanations, pseudocode, and static diagrams. Although these approaches help learners understand the overall logic and expected output, they often fail to demonstrate intermediate execution steps such as comparisons, swaps, recursive calls, pointer movements, and control flow decisions. As a result, students frequently struggle to visualize how algorithms manipulate data during execution, leading to gaps in conceptual understanding. Algorithm visualization has emerged as an effective educational approach by enabling learners to observe algorithm execution step by step. However, many existing visualization tools rely on simulated animations rather than executing real program code, limiting their ability to accurately represent actual runtime behavior.

To address these limitations, the **Master Algorithm** platform adopts a real-execution-based visualization approach in which algorithms are implemented in the **C programming language** and executed natively while runtime states are captured as structured execution traces. The enhanced platform supports **100 algorithms** across multiple domains and combines interactive visualization with **guided tutorials, MCQ-based learning checkpoints, integrated DAA theory, Interview Mode, customizable themes, and a categorized dashboard**. By integrating visualization, theoretical learning, and interview preparation within a single platform, the system provides a comprehensive learning environment suitable for academic courses, self-study, laboratory demonstrations, and technical interview preparation.

1.2 Problem Statement

To design and implement an interactive algorithm learning platform that executes real C-based algorithm implementations, captures step-by-step execution details, and visualizes them to improve learners' understanding of algorithm behavior. The platform also integrates guided tutorials, DAA theory, and interview preparation features to provide a comprehensive and interactive learning experience.

1.3 Objectives

The primary objectives of the **Master Algorithm: An Interactive Platform for Algorithm Execution Visualization** project are as follows:

1. To visualize algorithm execution step by step by capturing and displaying intermediate runtime states.
2. To improve conceptual understanding of algorithms and data structures through interactive visualization.
3. To execute authentic algorithm implementations written in the C programming language instead of simulated behavior.
4. To provide interactive execution controls such as play, pause, step, reset, and speed adjustment for self-paced learning.
5. To support learning through guided tutorials, integrated DAA theory, and interactive MCQ-based checkpoints.
6. To assist students in technical interview preparation through a dedicated Interview Mode and comprehensive algorithm coverage.
7. To provide a modular, categorized, and extensible platform supporting **100 algorithms** across multiple domains.
8. To ensure a secure, accessible, and user-friendly learning environment through input validation, customizable themes, keyboard navigation, and execution safeguards.

2. LITERATURE REVIEW

Recent research in computer science education has demonstrated that algorithm visualization significantly improves learner comprehension by making abstract execution behavior observable. Visualization-based learning environments enable students to understand how algorithms manipulate data structures and control flow during execution. However, existing studies also identify several shortcomings in current visualization tools, particularly with respect to execution accuracy, extensibility, and pedagogical effectiveness.

Hendriks *et al.* proposed a trace visualization framework for embedded and cyber-physical systems, illustrating that structured execution traces provide a precise representation of runtime behavior. Although the framework is not designed specifically for educational purposes, it establishes that trace-based representations are effective in analyzing dynamic execution processes, thereby supporting their use in algorithm visualization platforms.[1]

Avhad *et al.* emphasized the importance of step-by-step visual feedback and learner interaction in algorithm education. Their study reported improved comprehension when learners were able to observe algorithm execution dynamically. However, the visualization approach relied on simulated behavior rather than real code execution, limiting its ability to reflect authentic runtime characteristics.[2]

The iFlow system introduced by Ye *et al.* presented an interactive visualization tool for Max-Flow and Min-Cut algorithms, allowing users to actively explore execution paths and observe real-time state changes. The results demonstrated that hands-on interaction enhances understanding of complex graph algorithms. Nevertheless, the system is restricted to a narrow algorithm domain and does not generalize to broader algorithm categories.[3]

Execution-trace-based tools such as AbstractTrace, proposed by Al-Sharif and Jeffery, demonstrated how runtime events can be captured and analyzed to improve software testing and optimization. While primarily focused on software engineering applications, this work highlights the broader applicability of execution traces as a reliable method for understanding program behavior, reinforcing their relevance in educational visualization systems.[4]

Comparative visualization studies on sorting, searching, and traversal algorithms showed that animated and side-by-side visualizations help learners

identify performance differences and execution patterns more effectively. These findings underline the need for visualization platforms that are both extensible and capable of accurately representing algorithm behavior across multiple categories.[5]

Summary and Research Gaps:
The literature review highlights the following key gaps in existing algorithm visualization platforms:

1. **Limited Execution Accuracy:**
   Many existing tools rely on simulated animations instead of executing real algorithm implementations, resulting in an incomplete representation of actual runtime behavior.

2. **Restricted Algorithm Coverage:**
   Several platforms support only a limited set of algorithms, reducing their usefulness for comprehensive learning across different algorithm domains.

3. **Insufficient Learning Support:**
   Most visualization tools lack integrated tutorials, theoretical explanations, and interactive assessments that reinforce conceptual understanding.

4. **Limited Interview and Exam Preparation:**
   Existing systems primarily focus on visualization and do not provide features that help learners prepare for technical interviews or academic examinations.

5. **Accessibility and Customization Constraints:**
   Many platforms provide limited accessibility support and offer minimal customization options, affecting usability for diverse learners.

6. **Execution Reliability Issues:**
   Insufficient input validation and execution safeguards may lead to unstable or unreliable execution.

The Master Algorithm platform addresses these challenges by executing real C-based algorithm implementations and supporting **100 algorithms** across multiple categories. It combines guided tutorials, DAA theory, and Interview Mode.

3. SYSTEM ARCHITECTURE AND DESIGN

The proposed system follows a layered architectural approach designed to provide an interactive and comprehensive algorithm learning environment. It integrates user interaction, execution coordination, real algorithm execution, and educational support modules to enable step-by-step exploration of algorithm behavior. The architecture is structured to ensure modularity, scalability, maintainability, and efficient runtime execution.

Figure 3.1 illustrates the overall architecture of the **Master Algorithm** platform. The system is organized into multiple logical layers that separate user interaction, visualization, execution control, backend services, and algorithm implementation. This modular design enables the seamless integration of features such as guided tutorials, Interview Mode, DAA theory, categorized dashboards, and support for **100 algorithms**, while maintaining flexibility for future enhancements.

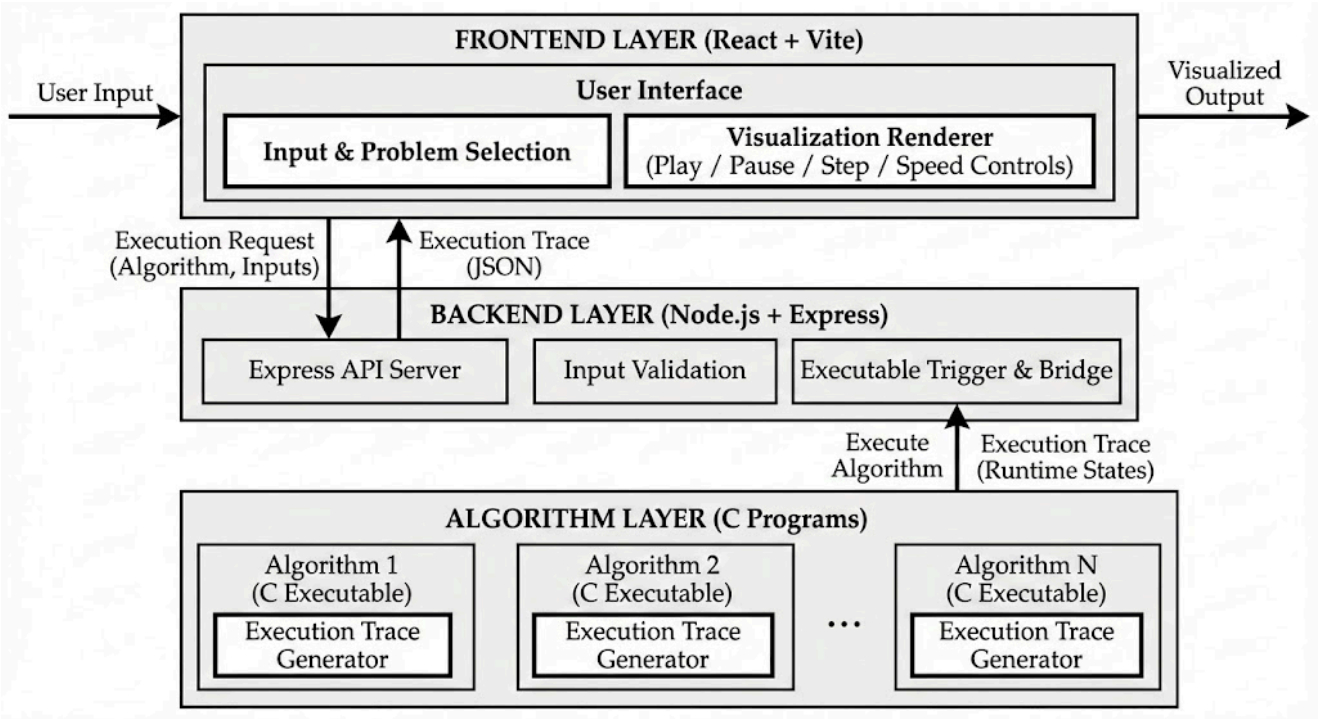

Fig. 3.1: System Architecture of the Algorithm Visualization Platform

3.1 Overall System Flow

The system operation begins when the user selects an algorithm and provides the required input through the web-based user interface. The frontend processes the input and sends an execution request to the backend execution controller. Upon receiving the request, the backend validates the input parameters and identifies the corresponding algorithm implementation.

The selected algorithm is then executed using a native C program, during which significant runtime events are captured as structured execution traces. These traces are transmitted back to the frontend, where they are rendered as step-by-step visualizations. This flow ensures that the visualization accurately reflects real algorithm execution while maintaining smooth and interactive user control.

3.2 User Interaction Layer

The user interaction layer is responsible for enabling intuitive and controlled interaction with the system. It provides interface elements that allow users to select algorithms, supply input values, and control execution using actions such as play, pause, step, reset, and speed adjustment.

This layer ensures that learners can explore algorithm behavior at their own pace, preventing automatic execution that could obscure intermediate steps. Accessibility features such as keyboard navigation and semantic interface design further enhance usability for a diverse set of learners.

3.3 Visualization Layer

The visualization layer is responsible for rendering the execution of algorithms based on the received execution traces. Each execution step is visually represented to reflect changes in data structures, control flow, and intermediate states during runtime.

Only the active elements involved in a particular execution step are highlighted at a time, minimizing visual clutter and improving clarity. This design allows learners to focus on the current operation while maintaining an understanding of overall execution flow.

3.4 Execution Control Layer

The execution control layer manages the sequencing and synchronization of algorithm execution. Execution steps are processed sequentially to ensure that each operation is visualized independently and clearly.

This layer enforces strict execution order and prevents overlapping steps, ensuring that the visualization remains consistent with actual runtime behavior. Controlled execution timing enables step-by-step exploration and reinforces understanding of algorithm flow and decision-making.

3.5 Algorithm Execution Layer

The algorithm execution layer consists of standalone implementations written in the C programming language. Each algorithm is executed independently to ensure correctness and transparency. During execution, key runtime events such as comparisons, swaps, recursive calls, and traversal steps are recorded.

These events are structured into execution traces that form the basis for visualization. By relying on real algorithm execution rather than simulated logic, the system ensures authenticity and accuracy in representing algorithm behavior.

3.6 Architectural Advantages

The proposed architecture offers the following advantages:

- Clear separation between visualization, execution control, and algorithm logic
- Accurate visualization based on real algorithm execution
- Modular design supporting easy addition of new algorithms
- Controlled and sequential execution ensuring clarity
- Improved learning effectiveness through interactive exploration

4. IMPLEMENTATION AND WEB INTERFACE

This section elaborates on the implementation methodology adopted for the algorithm execution visualization system and discusses the design of the interactive user interface. The approach prioritizes accurate runtime behavior, structured execution control, and clarity of visual representation.

4.1 Tools and Technologies

The proposed system is implemented using widely adopted and reliable tools suitable for web-based interactive visualization and native algorithm execution. Table 4.1 summarizes the major tools and technologies used in the development of the system.

Category	Tool / Technology	Purpose
Frontend Framework	React	User interface development
Build Tool	Vite	Fast development and build process
Backend Framework	Node.js with Express	Execution coordination and API handling
Programming Language	C	Algorithm implementation
Data Exchange Format	JSON	Execution trace representation
Build Tools	GCC	Compilation of C programs
Package Manager	npm	Dependency management
Version Control	Git	Source code management

Table 4.1: Tools and Technologies Used

4.2 Conceptual Algorithm Visualization Design

The conceptual visualization design aims to introduce algorithm behavior at a high level before exposing detailed execution steps. At this stage, algorithms are represented using abstract visual elements such as array blocks, nodes, or structural components, depending on the algorithm category. This abstraction prioritizes clarity and reduces cognitive load for beginner learners.

The conceptual design focuses on illustrating the overall flow of the algorithm, including initialization, iteration, and termination stages, without exposing low-level runtime details. This approach enables learners to first understand what the algorithm does before exploring how it executes internally.

Figure 4.2.1 illustrates the conceptual visualization layout used to represent algorithm structures in their initial state.

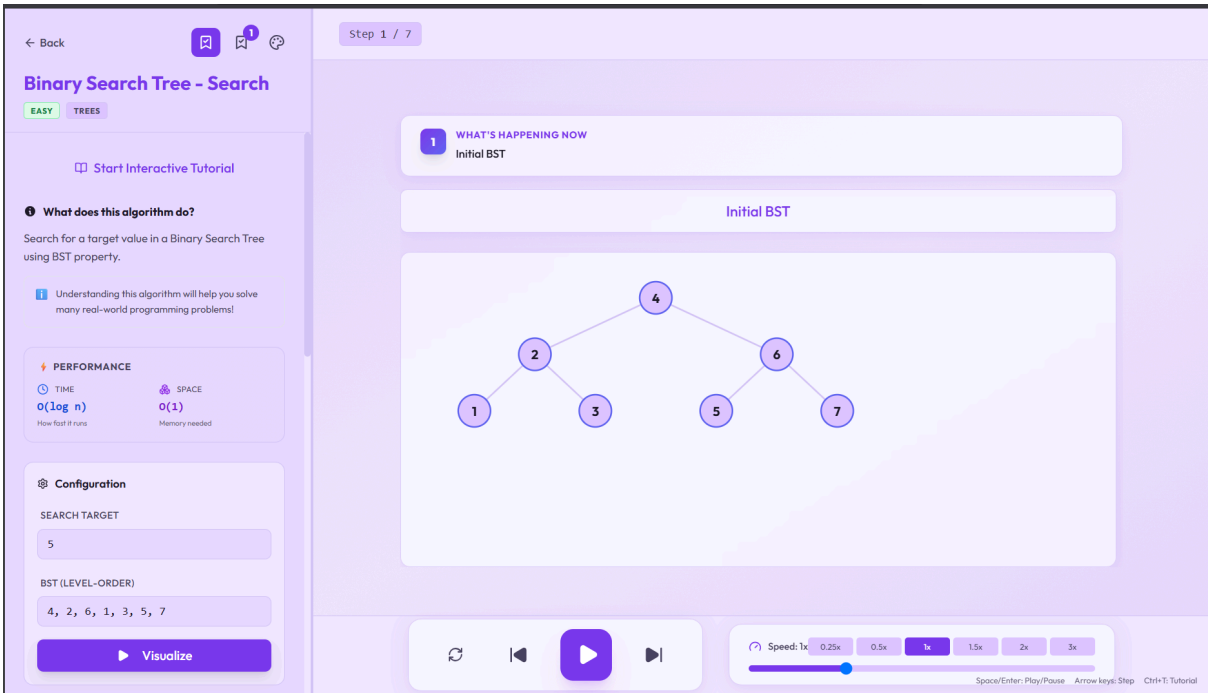

Figure 4.2.1: Conceptual Algorithm Visualization Design

4.3 Runtime Execution Visualization

During runtime, the visualization dynamically responds to user interaction and execution progress. When an algorithm is executed, each significant operation—such as a comparison, swap, recursive call, or traversal step - is visualized sequentially based on the execution trace received from the backend.

Only the currently active elements are highlighted at each step, while inactive elements remain unchanged. This selective highlighting minimizes visual clutter and allows learners to focus on the current operation. User-controlled execution features such as play, pause, step, reset, and speed adjustment enable detailed inspection of algorithm behavior at an individual step level.

Figure 4.3.1 shows the algorithm visualization during runtime execution, highlighting the active elements involved in a specific execution step.

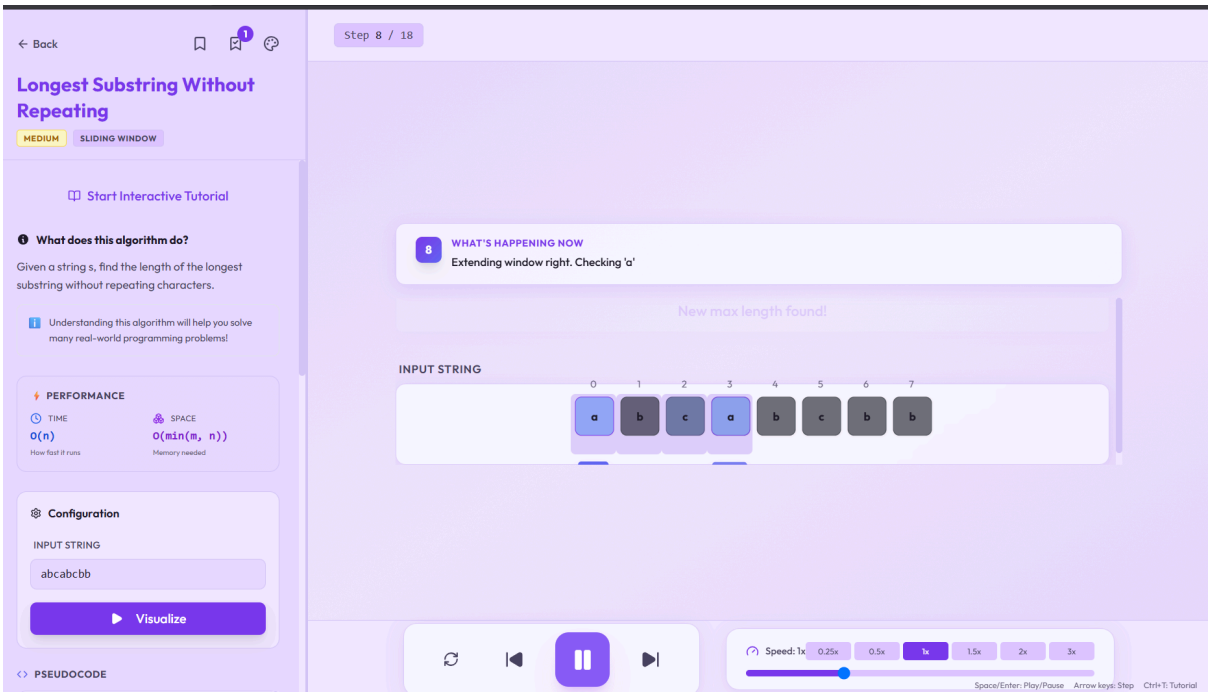

Figure 4.3.1: Algorithm Visualization During Runtime Execution

4.4 Algorithm Execution and Code-Level Implementation

Algorithm execution is carried out using standalone C implementations supporting **100 algorithms** across multiple domains. Each algorithm is executed natively, capturing runtime events such as comparisons, assignments, swaps, recursive calls, and traversal operations to ensure accurate, execution-driven visualization.

These runtime events form the basis for execution trace generation, ensuring a direct correspondence between the algorithm's implementation and its visual representation. Presenting the actual algorithm code alongside visualization strengthens the connection between source-level logic and observed runtime behavior.

4.5 User Interface Design

The user interface is designed to provide an intuitive and interactive learning experience through a modern, organized, and user-friendly layout. A categorized dashboard enables users to quickly browse algorithms across multiple domains, while sub-category navigation improves accessibility and ease of exploration.

The interface integrates algorithm visualization, source code display, complexity information, guided tutorials, DAA theory, and Interview Mode within a unified workspace. Interactive controls such as play, pause, step, reset, and speed adjustment allow users to explore algorithm execution at their own pace. Additional features, including customizable themes and accessibility support, further enhance usability and provide a personalized learning experience.

Figure 4.5.1 illustrates the complete user interface of the Master Algorithm platform.

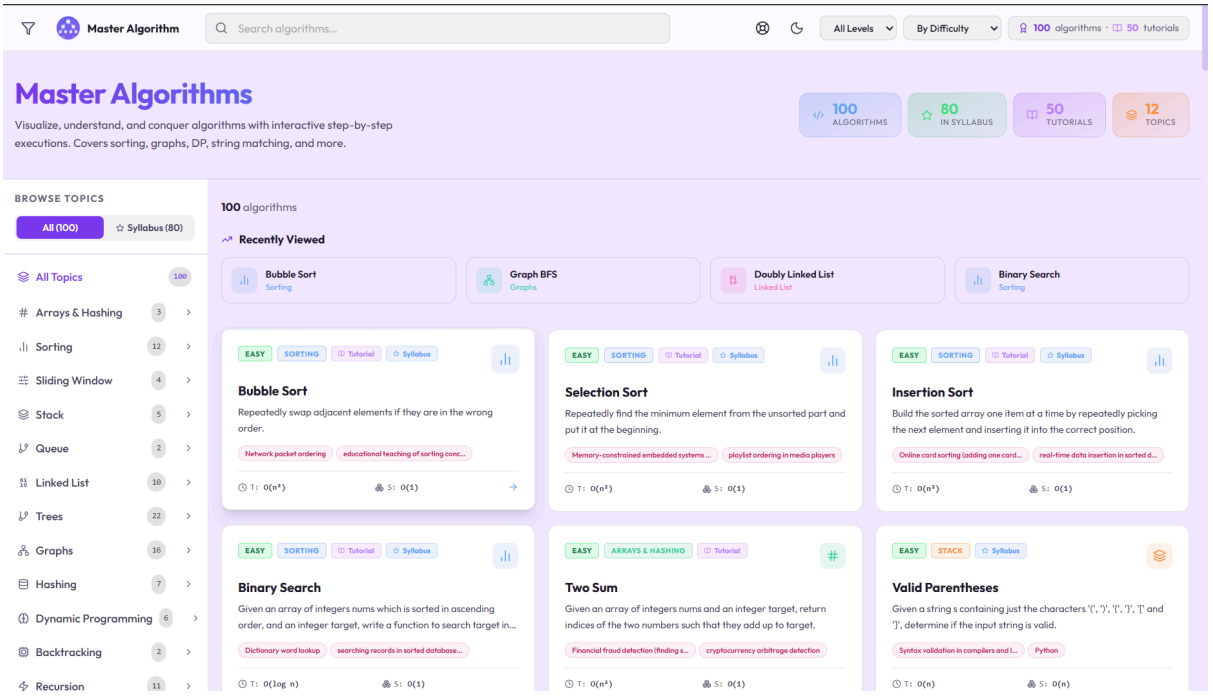

Figure 4.5.1: User Interface of Master Algorithm

4.6 Dashboard Features and Functional Modules

The system provides a unified dashboard that serves as the central learning environment for algorithm visualization and exploration. The redesigned dashboard integrates visualization, execution controls, theoretical learning resources, and interview preparation modules within a single interface. A categorized navigation system enables users to efficiently access algorithms from multiple domains while maintaining an intuitive and organized user experience.

The key features available through the dashboard are summarized as follows:

- **Algorithm Library:**
  Supports **100 algorithms** organized into categories and sub-categories, enabling users to quickly explore algorithms across multiple domains.

- **Input Configuration Panel:**
  Allows users to provide custom input values for executing and experimenting with different algorithms.

- **Execution Control Module:**
  Provides play, pause, step, reset, and speed adjustment controls for interactive, step-by-step execution.

- **Runtime Visualization Panel:**
  Displays real-time visualization of algorithm execution by highlighting intermediate operations and state changes.

- **Code and Complexity Panel:**
  Presents the corresponding C implementation along with time and space complexity to strengthen conceptual understanding.

- **Guided Learning Module:**
  Offers interactive tutorials with integrated MCQ-based checkpoints to reinforce learning.

- **DAA Theory Module:**
  Provides explanations of algorithm concepts, complexity analysis, recurrence relations, and other theoretical topics.

- **Interview Mode:**
  Helps users prepare for technical interviews through structured algorithm

practice and learning support.

- **Customization and Accessibility:**
  Supports multiple themes, keyboard navigation, and an organized interface to improve usability and accessibility.

Figure 4.6.1 illustrates the dashboard highlighting its major functional modules.

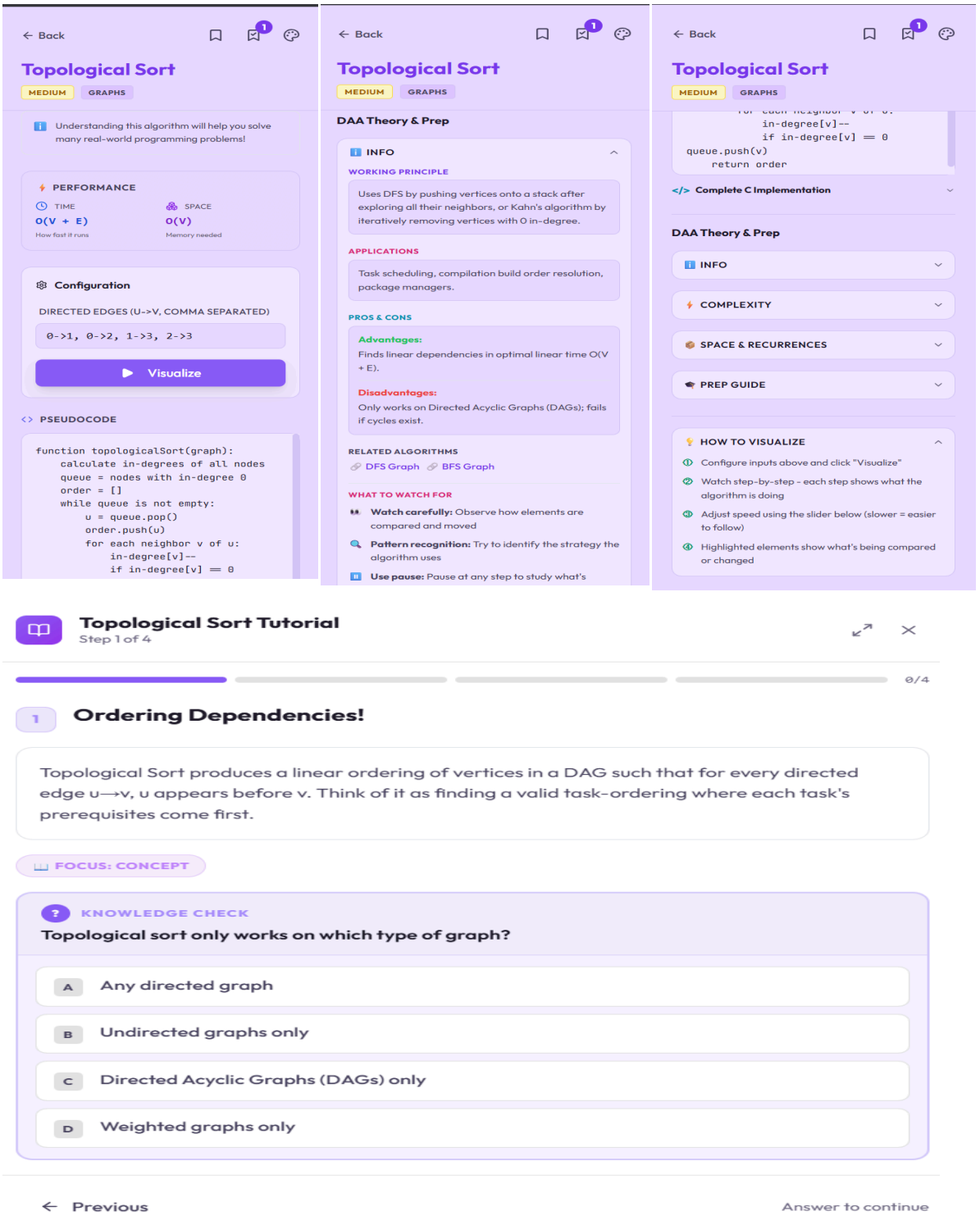

Fig. 4.6.1: Dashboard Showing Integrated Algorithm Visualization Features

4.7 Execution Control and Synchronization

Execution control logic ensures that algorithm steps are visualized sequentially and without overlap. Each execution trace entry is processed independently, maintaining a one-to-one correspondence between runtime operations and visual output.

Strict synchronization prevents multiple operations from being visualized simultaneously, ensuring clarity and correctness. This controlled execution model allows learners to clearly associate each visual change with the corresponding algorithmic operation, reinforcing conceptual understanding.

5. RESULTS AND DISCUSSION

5.1 Results

The developed algorithm execution visualization platform successfully delivers an interactive environment for understanding algorithm behavior through real execution. The system reliably supports algorithm selection, input configuration, and controlled execution through a unified dashboard interface. User interactions such as play, pause, step, reset, and speed adjustment function smoothly, allowing learners to explore algorithm execution without noticeable latency or instability.

At the conceptual level, the platform effectively introduces algorithm structure and overall execution flow, enabling users to understand initialization, iteration, and termination stages before exploring detailed runtime behavior. At the execution level, step-by-step visualization driven by execution traces generated from native C implementations accurately represents operations such as comparisons, swaps, recursive calls, and traversal steps. This execution-driven approach ensures that visual output corresponds directly to real algorithm behavior rather than simulated animations.

The integration of algorithm source code alongside visualization further strengthens understanding by linking theoretical concepts with practical implementation. The dashboard features, including code view and complexity information, function as intended and provide additional learning support. Overall, the system demonstrates stable performance and reliable execution, validating the correctness and effectiveness of the proposed approach.

5.2 Broader Applications

The proposed algorithm execution visualization platform can be applied in various academic and training contexts as described below:

1. Academic Teaching Support:
   The platform can be used as a supplementary teaching aid in data structures and algorithms courses to enhance conceptual understanding through interactive execution visualization.

2. Laboratory and Demonstration Use:
   The step-by-step execution control makes the system suitable for

laboratory sessions and classroom demonstrations where instructors can explain algorithm behavior interactively.

3. Self-Learning and Exam Preparation:
   Learners can independently explore algorithm execution, guided tutorials, and DAA theory at their own pace, making the platform suitable for self-study, revision, and academic exam preparation.

4. Interview Preparation:
   By visualizing algorithm logic and execution flow, the system assists students in strengthening problem-solving skills required for technical interviews.

5. Online and Blended Learning Platforms:
   The web-based nature of the system allows integration into online learning environments and educational portals.

6. Technical Training Programs:
   The platform can be adapted for use in workshops, coding bootcamps, and technical training programs to reinforce algorithm fundamentals.

5.3 Limitations

Despite its effectiveness, the proposed system has certain limitations as outlined below:

1. Focus on Conceptual Clarity:
   The visualization emphasizes algorithm logic and control flow rather than low-level details such as memory layout or hardware-specific behavior.

2. Lack of Performance Benchmarking:
   The system does not provide execution time analysis or performance comparison between algorithms.

3. Scalability of Visualization:
   Algorithms with large input sizes may result in dense or complex visual representations, requiring careful navigation by the user.

4. Educational-Oriented Design:
   The platform is designed primarily for learning purposes and is not intended for production-level algorithm analysis or optimization.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

The **Master Algorithm** platform successfully provides an interactive and execution-driven environment for learning algorithms through real-time visualization. By executing real C-based algorithm implementations and presenting step-by-step execution traces, the platform enables learners to understand algorithm behavior more effectively than traditional static learning methods.

The enhanced platform supports **100 algorithms** across multiple domains and integrates guided tutorials, DAA theory, Interview Mode, complexity analysis, and interactive execution controls within a unified learning environment. The redesigned dashboard, customizable themes, and accessibility features further improve usability and learner engagement.

Overall, the project successfully achieves its objectives by combining algorithm visualization, theoretical learning, and interview preparation into a single educational platform, making it valuable for students, educators, self-learning, and technical interview preparation.

6.2 Future Work

The proposed system can be further enhanced in the following ways:

1. Cloud Integration:
   Enable cloud-based execution and synchronization to improve scalability, accessibility, and collaborative usage.

2. Performance Analysis:
   Integrate execution time, memory usage, and comparative performance metrics to help learners analyze algorithm efficiency.

3. Collaborative Learning Features:
   Introduce user progress tracking, quizzes, and instructor dashboards to support classroom learning and personalized assessment.

4. AI-Assisted Learning:
   Incorporate AI-powered explanations and personalized recommendations to provide adaptive learning support.

REFERENCES

[1] M. Hendriks, J. Verriet, and T. Basten, “Visualization, transformation, and analysis of execution traces with the Eclipse Trace4CPS trace tool,” International Journal on Software Tools for Technology Transfer, vol. 26, pp. 101–126, Feb. 2024, doi: 10.1007/s10009-024-00736-3.

[2] P. Avhad, A. Sawant, V. Lakhe, and G. T. Avhad, “Algorithm visualization for educational and analytical purpose,” International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET), vol. 13, no. 5, May 2024, doi: 10.15680/IJIRSET.2024.1305389.

[3] M. Ye et al., “iFlow: An interactive max-flow/min-cut algorithms visualizer,” arXiv preprint, arXiv:2411.10484, Nov. 2024.

[4] Z. A. Al-Sharif and C. L. Jeffery, “AbstractTrace: The use of execution traces to cluster, classify, prioritize, and optimize a bloated test suite,” Applied Sciences, vol. 14, no. 23, Art. no. 11168, 2024, doi: 10.3390/app14231168.

[5] S. Rathod, “Visualization and comparative simulation of pathfinding, searching and sorting algorithms,” Journal of Emerging Engineering Technologies (JEET), 2024.

[6] Ministry of Education, Government of India, “Virtual Labs: Data Structures Laboratory,” IIIT Hyderabad. [Online]. Available: https://ds1-iiith.vlabs.ac.in/. Accessed: Jul. 4, 2026.

[7] Ministry of Education, Government of India, “Virtual Labs: Design and Analysis of Algorithms Laboratory,” Dayalbagh Educational Institute. [Online]. Available: https://daa-dei.vlabs.ac.in/. Accessed: Jul. 4, 2026.

[8] R. Osztian, A. Korhonen, and T. Myller, “Investigating the Effect of Interactivity in Algorithm Visualization,” in Proc. IEEE Frontiers in Education Conference (FIE), Uppsala, Sweden, 2020, pp. 1–9, doi: 10.1109/FIE44824.2020.9274065.

[9] J. Sorva and V. Karavirta, “Engagement and learning with algorithm visualization systems: A systematic review,” ACM Transactions on Computing Education, vol. 23, no. 2, Art. no. 15, Apr. 2023, doi: 10.1145/3572842.

[10] T. Naps, G. Rößling, V. Almstrum, W. Dann, R. Fleischer, C. Hundhausen, A. Korhonen, L. Malmi, M. McNally, and S. Rodger, “Exploring the role of visualization and engagement in computer science education,” Communications of the ACM, vol. 66, no. 7, pp. 54–61, Jul. 2023, doi: 10.1145/3581785.

[11] C. Hundhausen, S. Douglas, and J. Stasko, “A meta-study of algorithm visualization effectiveness,” Journal of Visual Languages and Computing, vol. 76, Art. no. 101948, 2024, doi: 10.1016/j.jvlc.2023.101948.

[12] A. Korhonen and L. Malmi, “Algorithm simulation with interactive prediction tasks,” IEEE Transactions on Education, vol. 67, no. 1, pp. 12–20, Feb. 2024, doi: 10.1109/TE.2023.3324189.
[13] R. Bednarik, M. Tukiainen, and T. Myller, “The role of cognitive load in program and algorithm visualization,” Computers & Education, vol. 201, Art. no. 104820, 2023, doi: 10.1016/j.compedu.2023.104820.

ACKNOWLEDGEMENTS

We express our sincere gratitude to **Prof. Ganashree K. C**, Assistant Professor, for her valuable guidance, continuous support, and constructive feedback throughout the course of this work. Her insights and encouragement were instrumental in shaping the direction and successful completion of the project.

We also acknowledge the use of open-source tools and resources that supported the development of this project. The complete implementation and related materials are available in the project repository.

https://github.com/Avi007-debug/Master_Algorithm

Following are the additional references that also contributed in research that went into making of the platform :

Ref	Paper	Methodology	Key Findings	Limitations
[1]	IIT Virtual Labs (IIIT Hyderabad & DEI)	Interactive browser-based simulations with tutorials and quizzes	Improves conceptual understanding through guided experiments	Uses simulated execution instead of real program execution
[2]	Osztian et al., *AlgoRythmics* (2020)	Interactive algorithm animations with prediction-based learning	Active learner engagement improves learning more than passive visualization	Limited algorithm support and predefined animations